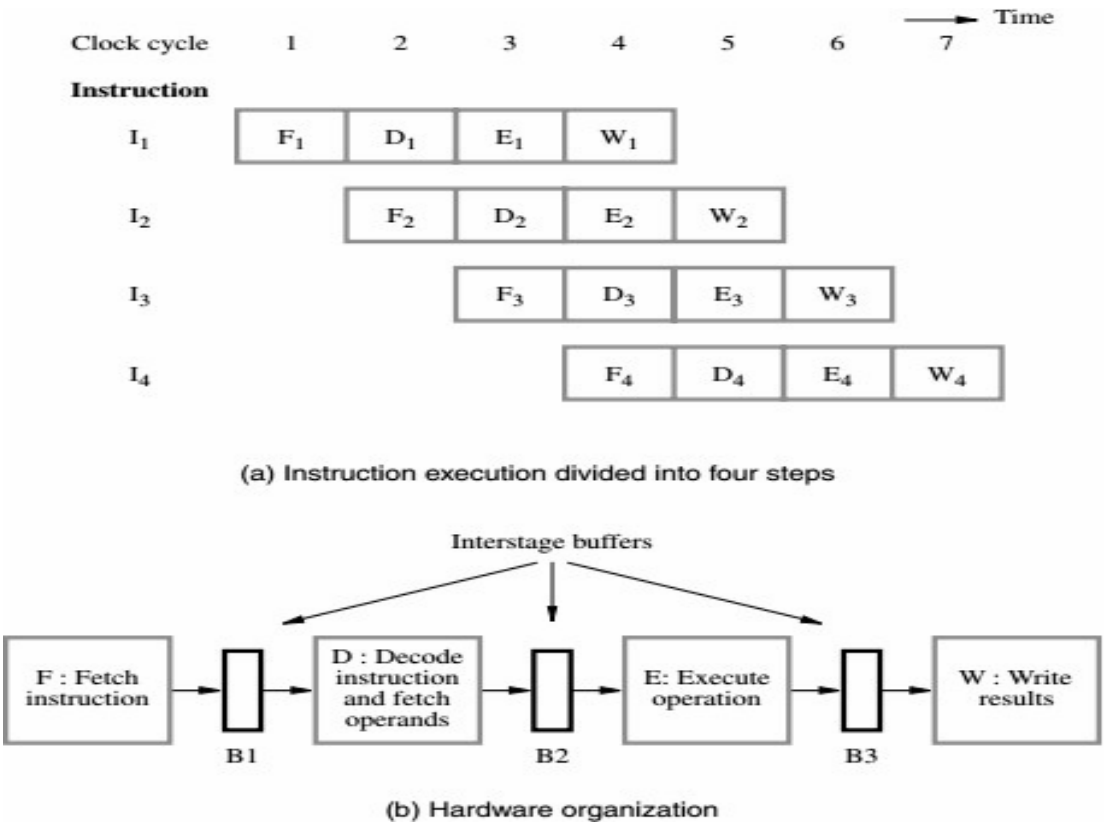
Pipelining:

An implementation technique in which multiple instructions are overlapped in execution iscalled pipeline. The different pipelining stages are,

1. **Fetch** - Fetch instruction from memory.
2. **Decode** - Read registers while decoding the instruction. The regular format of MIPS instructions allow reading and decoding to occur simultaneously.
3. **Execute** - Execute the operation or calculate an address.
4. **Access** - Access an operand in data memory.
5. **Write** - Write the result into a register.

Four stage Instruction Pipelining



Hardware units are organized into stages:

- Execution in each stage takes exactly 1 clock period. Stages are separated by pipelineregisters that preserve and pass partial results to the next stage.

Performance = complexity + cost.

- The pipeline approach brings additional expense plus its own set of problems andcomplications, called hazards.

Pipeline Performance (or) Speedup

- The potential increase in performance resulting from

pipelining is proportional to thenumber of pipeline stages.

- If all the stages take about the same amount of time and there is enough work to do,then the speed-up due to pipelining is equal to the number of stages in the pipeline.
- If the stages are perfectly balanced, then the time between instructions on thepipelined processor –

$$\text{Time between instructions}_{\text{pipelined}} = \frac{\text{Time between instruction}_{\text{nonpipelined}}}{\text{Number of pipe stages}}$$

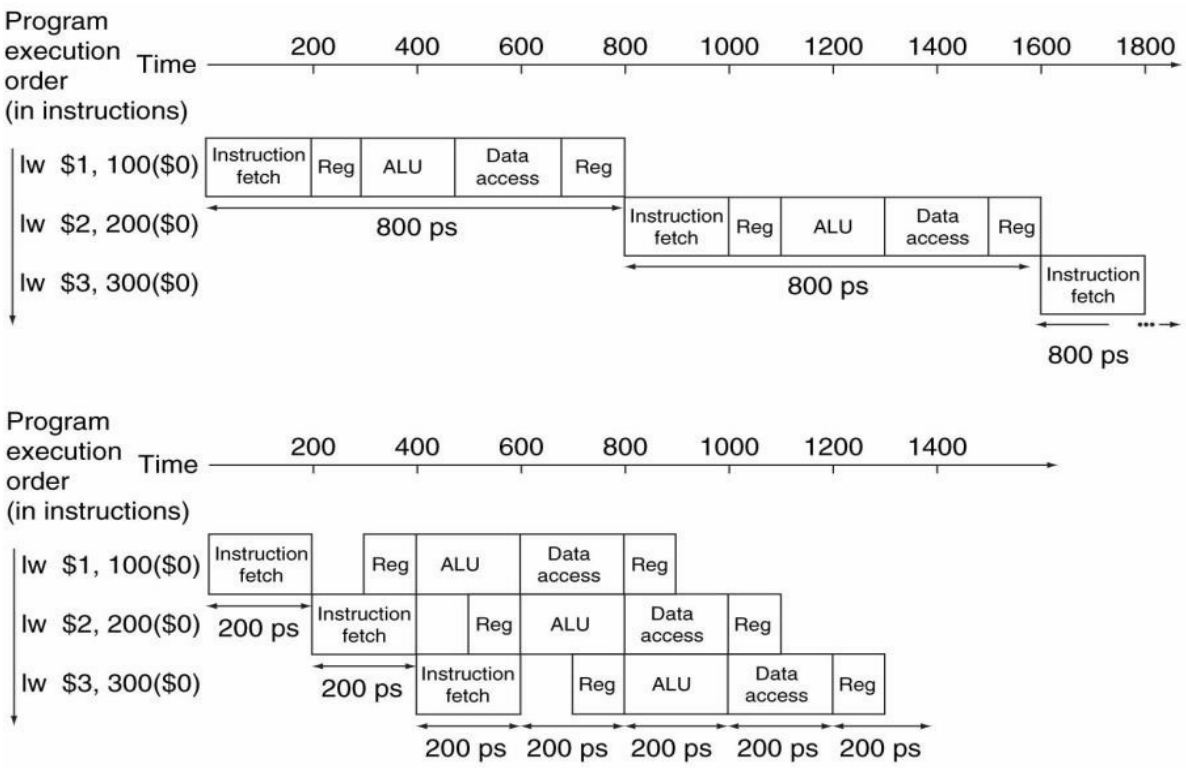
assuming ideal conditions – is equal to

- A pipelined processor allows multiple instructions to execute at once, and each instruction uses a different functional unit in the datapath. This increases throughput, so programs can run faster. One instruction can finish executing on every clock cycle, and simpler stages also lead to shorter cycle times.

Example – Single-Cycle versus Pipelined Performance

- Consider a simple program segment consists of eight instructions: lw, sw, add, sub, AND, OR, slt and beq. Compare the average time between instructions of a single- cycle implementation, in which all instructions take 1 clock cycle, to a pipelined implementation. The operation times for the major functional units in this example are200 ps for memory access, 200 ps for ALU operation, and 100 ps for register file read or write.
- The following table shows the time required for each of the eight instructions. The single-cycle design must allow for the slowest instruction is lw – so the time required for every instruction is 800 ps. Thus, the time between the first and fourth instructions in the non-pipelined design is 3 x 800 ns or 2400 ps.

Instruction class	Instruction fetch	Register read	ALU operation	Data access	Register write	Total time
Load word (`lw`)	200 ps	100 ps	200 ps	200 ps	100 ps	800 ps
Store word (`sw`)	200 ps	100 ps	200 ps	200 ps		700 ps
R-format (`add`, `sub`, `AND`, `OR`, `slt`)	200 ps	100 ps	200 ps		100 ps	600 ps
Branch (`beq`)	200 ps	100 ps	200 ps			500 ps



- Assume that following table shows the time taken by each and every stages of pipeline for different instruction

Figure: Single-Cycle, Non-Pipelined Execution in top versus Pipelined Execution in bottom.

- By comparing above two diagram, it is clear that pipeline process is best and it take reduce time to execute the instruction.
- Pipelining improves performance by increasing

instruction throughput, as opposed to decreasing the execution time of an individual instruction.

Six stages in the pipeline:

1. Fetch instruction: Instructions are fetched from the memory into a temporary bufferbefore it gets executed.

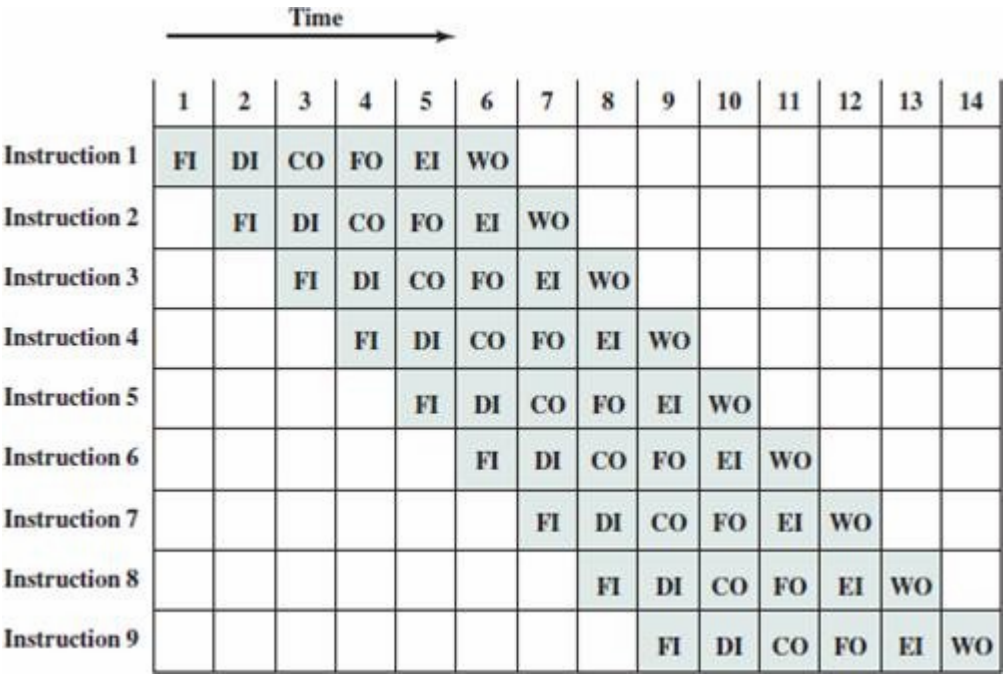
2. Decode instruction: The instruction is decoded by the CPU so that the necessary op codesand operands can be determined.

3. Calculate operand: Based on the addressing scheme used, either operands are directlyprovided in the instruction or the effective address has to be calculated.

4. Fetch Operand: Once the address is calculated, the operands need to be fetched from theaddress that was calculated. This is done in this phase.

5. Execute Instruction: The instruction can now be executed.

Write operand: Once the instruction is executed, the result from the execution needs to bestored or written back in the memory



Pipeline Hazards

The condition that makes the pipeline to stall is called Hazards. The idle period in thepipeline execution is called Stall or Bubble.

Types of hazards:

1. Structural Hazard
2. Data Hazard
3. Control Hazard

1. Structural Hazard

- When a planned instruction cannot execute in the proper clock cycle because the hardware does not support the combination of instructions that are set to execute.

2. Data Hazards

- **Data hazards** occur when the pipeline must be stalled because one step mustwait for another to complete.
- When a planned instruction cannot execute in the proper clock cycle becausedata that is needed to execute the instruction is not yet available.
- This is because of data dependence between the instructions that has beenoverlapped.

Consider the following example

```
add    $s0, $t0, $t1
sub    $t2, $s0, $t3
```

- In the above instruction one of the operand ($s0) of the sub instruction will befetched only after the add instruction store it result in the same register ($s0).
- So that sub instruction is stalled for some clock cycle which makes thepipeline process to waste the some clock cycle.

3. Control Hazards

- It is also called branch hazard. When the proper instruction cannot execute in the proper pipeline clock cycle because the instruction that was fetched is not the one that is needed; that is, the flow of instruction addresses is not what the pipeline expected.